Nome: Bianca Banaletti e Sabrina Figueiredo
Disciplina: Sistemas Operacionais II

Resumo da aula do dia 12/08

Mecanismos de sincronização: Os mecanismos que garantem a comunicação entre processos concorrentes e o acesso a recursos compartilhados são chamados mecanismos de sincronização. No projeto de sistemas operacionais multiprogramáveis, é fundamental a implementação destes mecanismos para garantir a integridade e a confiabilidade na execução de processos que compartilham recursos. Seu objetivo é evitar conflitos, garantir a exclusão mútua e fazer com que os processos executem determinadas ações na ordem ou condição correta.

Semáforos: São mecanismos de sincronização baseados em uma variável inteira não negativa, manipulada pelas operações DOWN e UP. O DOWN diminui o valor e, quando o semáforo está em 0, faz o processo esperar. O UP aumenta o valor e pode liberar um processo que estava esperando. Existem dois tipos:

- Binário: assume apenas 0 ou 1, sendo usado principalmente para exclusão mútua.
- Contador: pode assumir valores maiores que 1, sendo utilizado para controlar vários recursos disponíveis.

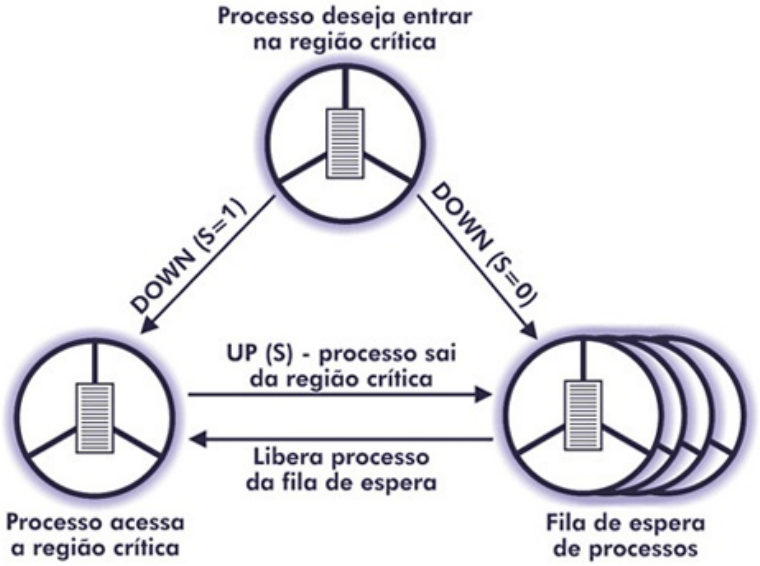


Fig. 7.3 Utilização do semáforo binário na exclusão mútua.

Mutex: O mutex é utilizado para garantir que apenas um processo por vez acesse uma região crítica ou recurso compartilhado. No semáforo binário, o valor 1 indica que o recurso está livre e 0 indica que está sendo utilizado. O processo executa DOWN para entrar na região crítica e UP para sair e liberar o recurso.

Monitores: São mecanismos de sincronização de alto nível e estruturados, criados para facilitar o desenvolvimento de programas concorrentes. Um monitor reúne variáveis e procedimentos e garante automaticamente a exclusão mútua, permitindo que apenas um processo execute um procedimento do monitor por vez. Para sincronização condicional, utilizam:

- WAIT: coloca o processo em espera até que uma condição seja atendida.
- SIGNAL: sinaliza que a condição foi atendida e libera um processo que estava esperando.

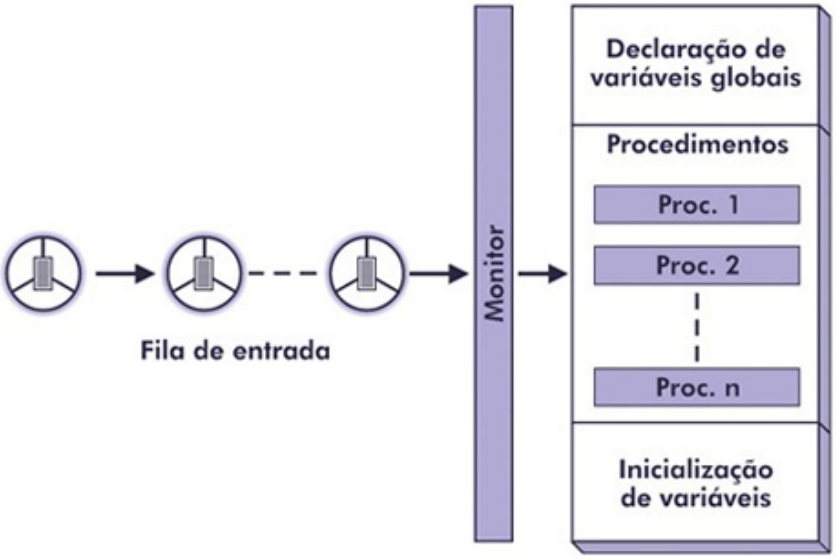


Fig. 7.4 **Estrutura do monitor.**

Problema dos filósofos: É um problema clássico de sincronização em que cinco filósofos precisam de dois garfos para comer. O problema ocorre quando cada filósofo pega um garfo e fica esperando pelo segundo. Nesse caso, todos podem ficar esperando indefinidamente, causando um deadlock. Algumas formas de evitar o deadlock são limitar a quantidade de filósofos sentados, permitir que um filósofo pegue os dois garfos somente quando ambos estiverem disponíveis ou fazer filósofos pares e ímpares pegarem os garfos em ordens diferentes.

Problema do barbeiro: Representa uma situação em que um barbeiro atende clientes, existindo uma cadeira para o atendimento e cinco cadeiras de espera. Se houver espaço, o cliente aguarda; se todas as cadeiras estiverem ocupadas, ele vai embora. Quando não há clientes, o barbeiro fica esperando até que um cliente apareça. O problema utiliza semáforos para controlar os clientes, o barbeiro e o acesso à variável que indica as cadeiras ocupadas. Assim, os processos conseguem trabalhar de forma sincronizada sem acessar os recursos de maneira conflitante.